Reviewer Pack — Minimal Rank of Quaternion Algebras vs. Quantum Circuit Dept...

Entry c8f4e8395379 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 00:54:19 UTC

Minimal Rank of Quaternion Algebras vs. Quantum Circuit Depth for Clifford Group States

Entry ID: `c8f4e8395379` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 00:54:19 UTC

1. Conjecture

Field A (mathematical branch): Quaternion Algebra Theory **Field B** (complexity object): Quantum Information Theory: Circuit Complexity (Clifford Group States)

Statement:

> {'quantitative_relation': 'For all n-bit quantum circuits C that implement the Clifford group, the minimal rank of the associated quaternion algebra Q(C) is Θ(n^2 log n).', 'counterexample_statement': 'There exists a quantum circuit C with depth O(log n) that implements a Clifford group for which the minimal rank of its associated quaternion algebra is o(n^2 log n).'}

Rationale (proposer's reasoning):

> {'explanation': 'Quaternion algebras have been used to represent geometric structures, and their ranks can reveal information about the complexity of transformations. Quantum circuits that implement Clifford groups are fundamental in quantum computing, with deep connections to complexity theory. This conjecture proposes a connection between algebraic properties and circuit complexity.', 'link_to_complexity': 'If true, it would suggest a way to lower bound the complexity of certain quantum circuits using algebraic techniques.'}

Taxonomy category: `QUANTUM_ALGEBRA_CONNECTION` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1027d32c8d4ad896`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n-bit Clifford group circuits, the minimal rank of the quaternion algebra Q(C) is within a factor of 3 from Θ(n^2 log n), with an average across at least 10 seeds; it is falsified if any seed produces a minimal rank significantly below o(n^2 log n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank quaternion algebra" AND "quantum circuit complexity" - "Clifford group states" AND "quaternion algebra" - "θ(n^2 log n) minimal rank" AND "Clifford group quantum circuits"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for k in range(i+1, n):
            if abs(matrix[k][i]) > abs(matrix[max_row][i]):
                max_row = k
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below the pivot
        factor = Fraction(matrix[i][i])
        for j in range(i, n):
            matrix[i][j] /= factor

        for k in range(i+1, n):
            factor = Fraction(matrix[k][i])
            for j in range(i, n):
                matrix[k][j] -= factor * matrix[i][j]

    # Back substitution
    result = [0] * n
    for i in range(n-1, -1, -1):
        result[i] = matrix[i][-1]
        for j in range(i+1, n):
            result[i] -= matrix[i][j] * result[j]
        result[i] /= Fraction(matrix[i][i])
```

```
    return result

def matrix_rank(matrix):
    n = len(matrix)
    m = len(matrix[0])
    rank = 0
    for i in range(n):
        if all(matrix[i][j] == 0 for j in range(m)):
            continue
        rank += 1
        factor = Fraction(1, matrix[i][i])
        for j in range(m):
            matrix[i][j] *= factor

        for k in range(n):
            if k != i:
                factor = Fraction(matrix[k][i], matrix[i][i])
                for j in range(m):
                    matrix[k][j] -= factor * matrix[i][j]
    return rank

def clifford_group_circuit(n):
    # Placeholder function to generate a random n-bit Clifford group circuit
    # This is a stub and should be replaced with actual quantum circuit generation logic
    return [[random.choice([0, 1]) for _ in range(2*n)] for _ in range(2**n)]

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [10, 20, 40]
    results = []

    for n in n_values:
        circuit = clifford_group_circuit(n)
        Q_C = [[random.choice([0, 1]) for _ in range(2*n)] for _ in range(2*n)]
        rank = matrix_rank(Q_C)

        expected_rank = n**2 * Fraction(1, 2) * math.log2(n)
        if rank < expected_rank:
            return {
                "metric_name": "average_rank",
                "metric_value": rank,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"Rank {rank} is less than 0(n^2 log n) for n={n}"
            }

        results.append(rank)

    average_rank = sum(results) / len(results)
    return {
        "metric_name": "average_rank",
        "metric_value": average_rank,
        "instances_tested": len(results),
        "conjecture_holds": True,
```

```
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    import math

    if not sys.argv[1:]:
        seeds = [2**i + 7 for i in range(5, 8)]  # First 30 prime numbers
    else:
        seeds = list(map(int, sys.argv[1:]))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Rank below 0(n^2 log n)' first_failing_seed={fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9432530e.py", line 115, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9432530e.py", line 81, in run_trial
    rank = matrix_rank(Q_C)
           ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9432530e.py", line 56, in matrix_rank
    factor = Fraction(1, matrix[i][i])
             ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(1, 0)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified according to the pre-registered criteria. | next: Re-run the test without errors to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12731
2	propose	ollama_remote	glm4:latest	0	0	10194
3	preregistration	ollama_remote	glm4:latest	0	0	6127
4	novelty	ollama_remote	glm4:latest	0	0	4659
5	novelty	ollama_remote	glm4:latest	0	0	6315
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13733
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10240
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9257
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11854
10	judge	ollama_remote	glm4:latest	0	0	12779

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 97889 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/c8f4e8395379.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c8f4e8395379.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c8f4e8395379.tar.gz` (if generated)